

Towards Continuous Validation of Customer Value

Helena Holmström Olsson

Department of Computer Science
Malmö University
Sweden

+46 702109797

helena.holmstrom.olsson@ma.se

Jan Bosch

Department of Computer Science and
Engineering

Chalmers University of Technology

Sweden

+46 733 664705

jan.bosch@chalmers.se

ABSTRACT

While close customer collaboration is highlighted as a distinguishing characteristic in agile development, difficulties arise in large-scale agile development where the product owner can no longer represent the different needs of a large customer base. While most companies use the role of a product owner to represent the customer base, experiences show that prioritizations that are made are far from optimal. Also, once the decision to develop a feature has been taken, companies stop to continuously validate if this feature adds value to the large customer base. As experienced in the case companies we work with, re-prioritization of feature content is difficult once development has started, resulting in R&D investments in development of features that have no proven customer value. In this paper, and based on our experiences from working with five B2B software development companies, we present a conceptual model in which qualitative and quantitative customer feedback techniques allow for continuous validation and re-prioritization of feature content. In this way, large-scale software development companies can significantly improve responsiveness to customers throughout the development cycle, while at the same time increase accuracy of their development efforts.

Categories and Subject Descriptors

• **Software and its engineering** ~ **Software creation and management** ~ **Software development process management** ~ **Software development techniques**

Keywords

Large-scale agile development, customer feedback techniques, continuous customer validation, feature re-prioritization, hypotheses, customer value.

1. INTRODUCTION

During the last decade, the vast majority of software development companies have adopted agile development practices. As one of the distinguishing characteristics of agile development, close customer collaboration allows for more frequent validation of functionality and the risk of developing functionality that is not appreciated by customers is reduced. Typically, and as advocated by many of the agile development methods [1, 2, 3], the role of the product owner acts as a proxy to customers to make sure that

the functionality that is developed is aligned with their needs.

However, difficulties arise in large-scale agile development where the product owner can no longer represent the needs of a large customer base. To cater for this, the product owner talks to a selected number of customers. Also, customer-specific teams are introduced [4]. While these approaches are indeed helpful, they don't allow for an accurate understanding of mass-market needs. In addition, there is a lack of mechanisms to re-prioritize the feature backlog once development has started [5]. Due to lack of mechanisms to validate feature value with a large customer base, the outcome of the pre-study is difficult to question and continuous re-prioritizations of feature content is scarce. As a consequence, software companies invest in developing features that were considered value adding in the pre-study phase, but have no proven customer value [5, 6].

In this paper, and based on on-going multiple case study in five large-scale software companies, we present a conceptual model that helps companies adopt a development approach in which focus shifts from early specification of requirements to continuous validation of customer value, and in which agile practices expand far beyond the development teams.

2. BACKGROUND

2.1 Large-scale Agile Development

For more than a decade, agile development methods have gained popularity and become widely recognized within the field of software engineering. The methods promise shorter time-to-market, as well as higher flexibility to accommodate changes in requirements and thereby, increase companies' ability to respond to evolving customer needs [7]. As recognized by Kettunen and Laanti [8], the concept of agile is multi-dimensional, and there are many reasons for companies to adopt agile ways-of-working. Typically, most companies introduce agile methods to increase the frequency in which they release new features and new products, and as a way to improve their software engineering efficiency.

A key stakeholder in most agile methods is the product owner. The product owner acts as a proxy to customers to help make sure that their needs are reflected in the development of new software functionality [3]. Part of the product owner responsibilities is to have a vision of what is to be built, and to convey that vision to the development team. This is done in part through the product backlog, which is a list in which potential new features for the product are prioritized, and which is used by the development teams when selecting what to build. The product owner is typically a lead user of the system or someone from marketing or product management who has a solid understanding of customers, the market, the competition and of future trends for the domain in which the system or product operates.

While agile methods originally evolved to meet the needs of small and co-located development teams [8], they have become attractive also to companies involved in large-scale development of software, and today there are several attempts such as Industrial XP and Scrum of Scrums aiming at scaling agile methods [9]. When developing software in a large-scale context, companies have to satisfy a large number of customers with different needs. Typically, these customers have many ideas on how the software product can serve their particular needs. In this context, the role of product management is to inventory these needs, to combine, merge and prioritize among these, and to present a roadmap with a set of requirements for the next release of the system. To do this, the collection and management of customer feedback is critical for the success of software systems and their evolution over time [10].

2.2 Customer Feedback Techniques

As recognized in previous research [11, 12, 13], companies apply a broad range of techniques to collect customer feedback in the process of software development. Typically, these techniques allow for customers to engage in problem definition, requirements engineering and system evaluation and validation. Already before development starts, and in the early stages of the development cycle, use cases, scenarios, prototyping, customer interviews, customer observations and surveys are common techniques for capturing customers' experiences and perceptions [11]. Likewise, alpha- and beta-testing techniques are efficiently used pre- and during development in order to continuously validate feature content with customers. Recently, and due to more and more products being connected to the Internet, the opportunity to collect post-deployment data from the products has significantly increased [14, 15]. Due to the online nature of these systems, data is continuously collected while customers use the systems, and the cost of collecting this data is low. As experienced in our previous research [12, 13], this has resulted in companies collecting huge amounts of product data reflecting product operation and performance. However, and as reported in our previous research [5, 12, 13], companies struggle with how to manage the data that is collected, and how to include customer feedback into their development processes. In our earlier research [5], we coined the term 'the open loop' problem denoting the situation in which there is no way for product management to validate if the features they prioritized in the pre-study phase are also the features that are used by customers and that generate the expected revenue to the company. As a result, most companies experience a situation in which the confirmation of the correctness of the prioritizations takes place only after the product has been deployed to its customers, and there is the risk that companies invest in development of features that have no proven customer value.

Recently, increasing attention has been put to continuous experimentation with customers [5, 14, 15, 16, 17]. Inspired by the 'Build-Measure-Learn' loop as a central concept within the Lean Startup community [18], a number of approaches emphasizing rapid customer evaluation of small product increments are emerging. While the concept is not new [19], it has become a reality also in B2B contexts and there are a number of examples in which large software-intensive companies apply experiment techniques, such as e.g. A/B testing, from the Web 2.0 domain to learn about product use and customer behaviors [15, 17, 20]. However, and as recognized in this study, although companies have mechanisms to collect large amounts of data from customers and from products, they lack a systematic approach in which they utilize different customer feedback techniques to

continuously validate customer value. Below, and based on our experiences in five B2B software companies, we identify the key problems they experience in relation to data collection, and we present a conceptual model that help them accomplish a more flexible and dynamic development environment in which features are continuously re-prioritized based on rapid customer feedback.

3. RESEARCH METHOD

This paper reports on a multiple case study [21] covering a period of 20 months (July 2013 – February 2015). Our study is based on a mix of qualitative interviews and workshops with company representatives in five software-intensive companies. Four of the companies are embedded systems companies developing physical products in which software is becoming increasingly important and where the majority of new product functionality lies in software. The companies represent the telecom domain, the automotive domain, the network camera domain and the pump manufacturer domain. The fifth company is a pure software company developing optimization software systems primarily for the airline industry. The five case companies are presented in Table 1 below.

Table 1. The five case companies involved in the study.

Case company	Domain
A	A provider of telecommunication systems and equipment, communications networks and multimedia solutions for mobile and fixed network operators.
B	A software company specializing in navigational information, operations management and optimization solutions.
C	A network video company offering products such as network cameras, video encoders, video management software and camera applications for video surveillance.
D	An equipment manufacturer developing, manufacturing and selling a variety of products within the embedded systems domain.
E	A pump manufacturer producing circulator pumps for heating and air conditioning, as well as centrifugal pumps for water supply.

In all companies, we continuously met with groups consisting of multiple roles such as e.g. product managers, project managers, product owners, chief architects and software developers. During our study, we conducted a series of group interviews in each company, and we also met in workshop sessions at which all companies were represented. Each group interview lasted for two hours and all discussions were held in English. The two researchers documented the interviews and after each interview we shared our notes. In addition to the group interviews, we organized workshops at each company site, as well as joint workshops to which all companies were invited. At these workshops, we had pre-defined themes that were discussed among all company representatives, and at some occasions we had company representatives give presentations to share their experiences related to customer feedback techniques and data collection practices.

4. CASE STUDY FINDINGS

Below, and based on the group interviews and workshop sessions, we summarize the challenges that we identified in the five case study companies. The challenges represent the main problems they experience in relation to collection and use of customer

feedback in their development of software in a large-scale development environment (Table 2).

Table 2. Challenges identified in the five case companies.

Problem identified:	Description:
The ‘open loop’ problem	The situation in which product management experience difficulties in getting accurate customer data. This leads to a situation in which decisions are taken based on opinions rather than customer data, resulting in R&D investments that are not aligned with customer needs.
Large amount of unused features	Due to limited mechanisms to monitor feature usage, our case companies are convinced that a large number of the features they develop are never used, and that investments are put on functionality that are not proven valuable to customers.
Wrong implementation of features	There are different ways in which features can be implemented. However, there is no efficient way in which the companies can continuously validate these alternatives with customers to decide which alternative that is the most appreciated one.
Requirements are seen as “truths”	A common view in all case companies is that requirements are regarded as “truths”. All companies experience difficulties in validating requirements also after development has started.
Lack of feature optimization	In the companies we study, the majority of the development effort is allocated to new feature development. As a result, time is spent on adding functionality instead of re-implementing features that don’t work well.
Misrepresentation of customers	In large-scale development of software, customer representation is difficult. Typically, and as reported in the interviews, the customers that “scream the loudest” get recognized while other customers get forgotten.
Lack of validation of feedback	Qualitative customer feedback is never validated in later stages, causing a situation in which vast amounts of development takes place although it has never been proven valuable to customers.
Large amounts of (useless) data	All companies have significant data available that could be used to inform their development efforts, but they are unable to capitalize on this data. While big data offers great potential, there is the risk of useless data if the wrong questions are asked.

5. THE ‘QCD’ MODEL: A CONCEPTUAL MODEL FOR CONTINUOUS VALIDATION OF CUSTOMER VALUE

In response to the challenges outlined above, and as experienced in the case companies, we developed a conceptual model called the ‘Qualitative/quantitative Customer-driven Development’ (QCD) model (Figure 1). Our model advocates a development approach in which requirements, instead of being frozen early in the development process, are viewed as hypotheses that need to be continuously validated throughout the development cycle to prove customer value. Validation is done by using qualitative and/or quantitative customer feedback techniques (CFT’s), with the main intention that the feature backlog is re-prioritized also in later stages of development. In this way, customer feedback is used capture changing customer needs that can potentially lead to re-

prioritization of features. Also, by continuously validate feature content with customers, the model supports an approach in which features with no proven customer value are abandoned, a situation which rarely happens in the companies we work with due to lack of customer data that supports such a decision. The QCD model is presented in Figure 1 below.

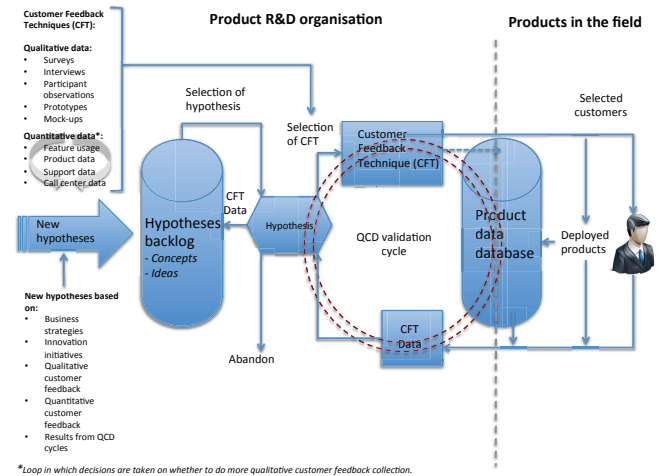


Figure 1. The QCD model for continuous validation of customer value

The QCD development approach is driven by hypotheses. The hypotheses are derived from business strategies, innovation initiatives, qualitative and quantitative customer feedback and results from on-going customer validation cycles. The hypotheses are concepts and ideas that are put into a hypotheses backlog with the intention to be validated with customers before being prioritized for development. Also, and as the distinguishing characteristic of the QCD approach, hypotheses are continuously validated also after development of features has started to allow for re-prioritizations of features during development. Once a hypothesis has been selected for validation, the company picks a customer feedback technique (CFT) for this purpose. Once a CFT has been chosen, the validation cycle starts. The validation cycle either involves a limited number of selected customers or it can be deployed directly in existing products in the field. If the CFT that was chosen is qualitative, the validation cycle consists of direct interactions with customers resulting in smaller amounts of qualitative data sets. If the CFT is quantitative, the validation cycle consists of having the feature deployed in products, instrument the code with metrics that monitor usage, collecting data revealing feature usage and store this in a product data database. The CFT data is then used to decide whether to re-prioritize the hypothesis and put it back.

The model was developed based on the challenges identified in the five case companies, and in our future research we aim to validate the model in the five case companies to learn more about how the model helps them solve the challenges identified.

6. CONCLUSIONS

In this paper, we present the ‘Qualitative/quantitative Customer-driven Development’ (QCD) model. The model is a conceptual model that was inductively derived from the challenges experienced in the five case companies. The model advocates an approach in which qualitative and quantitative customer feedback techniques are combined to run continuous validation cycles with customers throughout the development cycle. By advocating continuous validation of customer value, the model helps

companies adopt a development approach in which focus shifts from early specification of requirements to continuous validation of hypotheses, and in which agile practices expand far beyond the development teams.

7. ACKNOWLEDGMENTS

This study was funded by the Software Center at Chalmers University of Technology, University of Gothenburg and Malmö University in Sweden. We would like to thank all interviewees at the case companies for their time and engagement in this study.

8. REFERENCES

- [1] Highsmith, J., and Cockburn, A. 2001. Agile Software Development: The business of innovation, *Software Management*, September, pp. 120-122.
- [2] Larman, C. 2004. Agile and Iterative Development: A Manager's Guide. Addison-Wesley.
- [3] Schwaber, K., and Beedle, M. 2002. *Agile software development with Scrum*. Prentice Hall.
- [4] Olsson H.H., Sandberg A., Bosch J., and Alahyari H. 2014. Scale and responsiveness in large-scale software development. *IEEE Software*, Vol 31, Issue 5, pp. 87- 93.
- [5] Olsson, H.H., and Bosch, J. 2014. From Opinions to Data-Driven Software R&D: A Multi-Case Study On How To Close The 'Open Loop' Problem. In *Proceedings of EUROMICRO, Software Engineering and Advanced Applications (SEAA)*, August 27-29, Verona, Italy.
- [6] Backlund E., Bolle M., Tichy M., Olsson H.H., and Bosch J. 2014. Automated User Interaction Analysis for Workflow-Based Web Portals. In *Proceedings of the 5th International Conference on Software Business*, June 16-18, Paphos, Cyprus.
- [7] Williams L., and Cockburn A. 2003. Agile Software Development: It's About Feedback and Change. *Computer*, Vol. 36(6): 39-43.
- [8] Kettunen P., and Laanti M. 2008. Combining agile software projects and large-scale organizational agility. *Software Process Improvement And Practice*, Vol. 13: 183-193.
- [9] McMahon P. 2005. Extending agile methods: a distributed project and organizational improvement perspective. *CrossTalk*, Vol. 18(5): 16-19.
- [10] Neumann, M., Riel, A., and Brissaud, D. 2011. IT-supported innovation management in the automotive supplier industry to drive idea generation and leverage innovation, *Journal of Software Maintenance and Evolution: Research and practice*, Volume 301, 2012, pp. 229-240.
- [11] Bosch-Sijtsema, P., and Bosch, J. 2014. User involvement throughout the innovation process in high-tech industries. *The Journal of Product Innovation Management*. Online version published October 13.
- [12] Olsson H. H., and Bosch J. 2013. Towards Data-Driven Product Development: A Multiple Case Study on Post-Deployment Data Usage in Software-Intensive Embedded Systems. In *Proceedings of the Lean Enterprise Software and Systems Conference (LESS)*, December 1-4, 2013, Galway, Ireland.
- [13] Olsson H. H., and Bosch J. 2013. Post-Deployment Data Collection in Software-Intensive Embedded Products. In *Proceedings of the 4th International Conference on Software Business*, June 11-14, 2013, Potsdam, Germany.
- [14] Bosch, J. 2012. Building Products as Innovations Experiment Systems. In *Proceedings of 3rd International Conference on Software Business*, June 18-20, Cambridge, Massachusetts.
- [15] Bosch, J., and Eklund, U. 2012. Eternal Embedded Software: Towards Innovation Experiment Systems, In *Proceedings of International Symposium on Leveraging Applications*, 15-18 October, Crete.
- [16] Fagerholm, F., Sanchez G., Mäenpää, H., and Münch, J. 2014. Building blocks for continuous experimentation. In *the Proceedings of the RCoSE '14 workshop*, June 3, Hyderabad, India.
- [17] Kohavi, R., Crook, T., and Longbotham, R. 2009. Online Experimentation at Microsoft. In *the Third Workshop on Data Mining Case Studies and Practice Prize*.
- [18] Ries, E. 2011. *The Lean Startup: How Constant Innovation Creates Radically Successful Businesses*. London: Penguin Group.
- [19] Blank, S. 2005. The Four Steps to the Epiphany: Successful Strategies for Products that Win (3rd edition), Cafepress.com.
- [20] Olsson H. H., Alahyari H., and Bosch, J. 2012. Climbing the "Stairway to Heaven": A multiple-case study exploring barriers in the transition from agile development towards continuous deployment of software, In *Proceedings of the 38th Euromicro Conference on Software Engineering and Advanced Applications*, September 5-7, Cesme, Izmir, Turkey.
- [21] Runesson P., and Höst, M. 2009. Guidelines for conducting and reporting case study research in software engineering, *Empirical Software Engineering*, vol, 14.